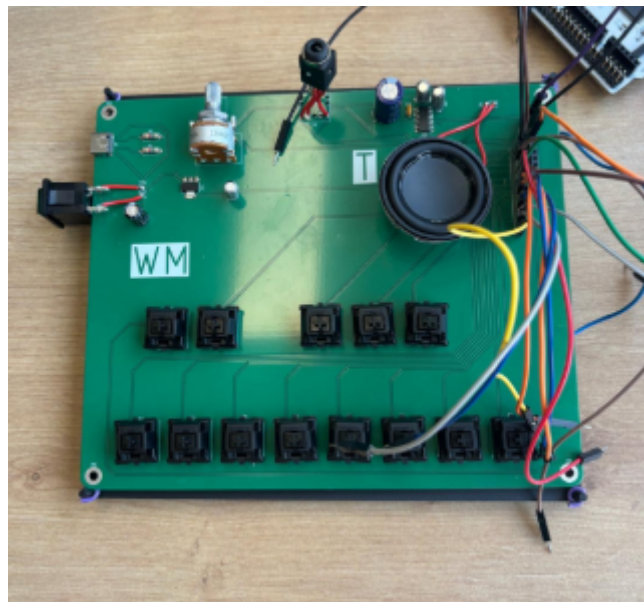


ENSEA — École Nationale Supérieure de l'Électronique et de ses Applications
Projet d'Électronique — 1ère Année — Semestre 6
Encadrant : M. Nicolas SIMOND | Salle C303

Piano Numérique

Développement d'un clavier — une octave

Conception, réalisation et validation d'un système embarqué



Groupe :

NORI Widad
CORBIÈRE Élie
FAUVEL Julien
ROUPENEL Léo
M'TIR Wissem
MESTRE PINCHO Claire

Période	Volume horaire
2 février 2026 – 28 mai 2026	40 h (9 × 4 h + soutenance) + 2 TD KiCad

Date de rendu : 28 mai 2026

Résumé

Ce projet présente la conception et la réalisation d'un piano numérique d'une octave (13 notes, Do à Do) développé autour de la carte Nucleo-L476RG embarquant un microcontrôleur STM32L476. La détection des touches repose sur treize switchs mécaniques Cherry MX, sélectionnés pour leur grande fiabilité matérielle et leur durabilité. La génération sonore est assurée en temps réel par l'algorithme de Karplus-Strong, initialement prototypé en Python pour valider le modèle mathématique, puis porté en langage C sur le microcontrôleur. Le signal audio est amplifié par un module PAM8403 (amplificateur Classe D, 3 W) et restitué via un haut-parleur intégré ou une prise Jack 3,5 mm selon les préférences de l'utilisateur. Le volume est réglable via un potentiomètre de 10 k Ω . L'alimentation est fournie par connecteur USB-C, avec régulation à 3,3 V assurée par un AMS1117-3.3. Le circuit imprimé, conçu sous KiCad en deux couches, a été fabriqué puis assemblé manuellement.

Mots-clés : piano numérique, Karplus-Strong, STM32L476, KiCad, PCB, DAC, Cherry MX, PAM8403, synthèse sonore temps réel, RSE.

Sommaire

Résumé

Introduction

I. Conception et Architecture Système

- I.1 Cahier des charges et diagramme SysML
- I.2 Bloc Acquisition — Détection des touches
- I.3 Bloc Traitement — Carte Nucleo-L476RG
- I.4 Bloc Alimentation — USB-C et régulation 3,3 V
- I.5 Bloc Sortie Audio — Amplification et sorties
- I.6 Tableau récapitulatif des composants

II. Réalisation Pratique

- II.1 Programmation
 - II.1.1 Modélisation mathématique en Python (Karplus-Strong)
 - II.1.2 Configuration des périphériques (STM32CubeMX)
 - II.1.3 Bibliothèque Karplus-Strong en C
 - II.1.4 Implémentation logicielle et simulation audio
- II.2 Conception électronique sur KiCad
 - II.2.1 Architecture du schéma électrique
 - II.2.2 Sélection des empreintes (footprints)
 - II.2.3 Placement des composants et ergonomie
 - II.2.4 Routage et plan de masse
- II.3 Soudage et Assemblage

III. Gestion de Projet, RSE et Sécurité

- III.1 Méthodologie et répartition des rôles
- III.2 Démarche RSE
- III.3 Analyse des risques et sécurité

Conclusion et Perspectives

Quatrième de couverture

Introduction

Dans le cadre du projet d'électronique de première année à l'ENSEA, notre groupe a choisi de concevoir un piano numérique d'une octave. Ce projet nous a permis de traverser le cycle de développement d'un système embarqué : rédaction du cahier des charges, sélection des composants, modélisation mathématique du son, développement logiciel en C sur microcontrôleur STM32, conception du circuit imprimé sous KiCad, et soudage manuel. Faute de temps, l'étape de validation par tests n'a pas pu être effectuée comme il aurait fallu.

Le choix de ce projet repose sur notre volonté de concevoir un système complet, interactif et fonctionnel. Un clavier numérique répond parfaitement à ces critères et mobilise des compétences en électronique analogique (amplification, filtrage), en électronique numérique (microcontrôleur, GPIO, DAC, timers), en traitement du signal (synthèse sonore, enveloppe temporelle) et en conception de PCB.

Le cahier des charges nous imposait des contraintes à la fois techniques et logistiques. Sur le plan technique, l'instrument devait produire les 13 notes d'une octave (du Do au Do) accordées sur le standard du La 440 Hz, avec une latence imperceptible à l'oreille et la capacité de jouer plusieurs notes simultanément (polyphonie). Sur le plan logistique, l'utilisation de la carte Nucleo-L476RG fournie par l'école était requise, et notre budget global était limité à 120 € (20 € par étudiant). Par ailleurs, nous avions initialement l'ambition de concevoir un clavier complet de quatre octaves. Cependant, les règles et contraintes de conception liées aux dimensions maximales autorisées pour la fabrication de notre PCB nous ont conduits à réduire la taille de l'instrument. Nous avons donc opté pour un clavier d'une seule octave.

Ce rapport s'articule en trois grandes parties. La première décrit la conception et l'architecture système, en justifiant les choix matériels pour chaque bloc fonctionnel. La deuxième détaille la réalisation pratique : algorithme de génération sonore, code embarqué C, conception PCB sous KiCad, et soudage. La troisième aborde la gestion de projet, la démarche RSE et l'analyse des risques.

I. Conception et Architecture Système

I.1 Cahier des charges et diagramme SysML

Avant de sélectionner les composants, nous avons formalisé les attentes du projet sous la forme d'un diagramme des exigences SysML. Cet outil nous a permis de rassembler visuellement l'ensemble des contraintes et des performances attendues, en les catégorisant en quatre types : contraintes matérielles (Hardware), contraintes budgétaires (Budget), exigences de performance (Précision, Latence, Étendue) et exigences fonctionnelles (Saisie_Touches, Polyphonie, Synthèse_Son).

Les principales exigences identifiées sont les suivantes. REQ1.1 : détection fiable de l'appui sur les 13 touches mécaniques. REQ1.2 : gestion de la polyphonie - permettre de jouer plusieurs notes simultanément. REQ1.3 : génération d'un signal audio via le DAC ou le PWM de la STM32. REQ1.4 : sortie audio sur port Jack ou haut-parleur externe. REQ2.1 : accordage précis basé sur le standard La 440 Hz. REQ2.2 : latence imperceptible entre l'appui mécanique et l'émission sonore. REQ3.1 : utilisation obligatoire du microcontrôleur STM32L476. REQ3.2 : coût des composants limité à 20 € par élève. REQ4.1 : recyclage des touches pour limiter l'impact environnemental. REQ4.2 : respect des règles de sécurité des installations et protection de la carte STM32.

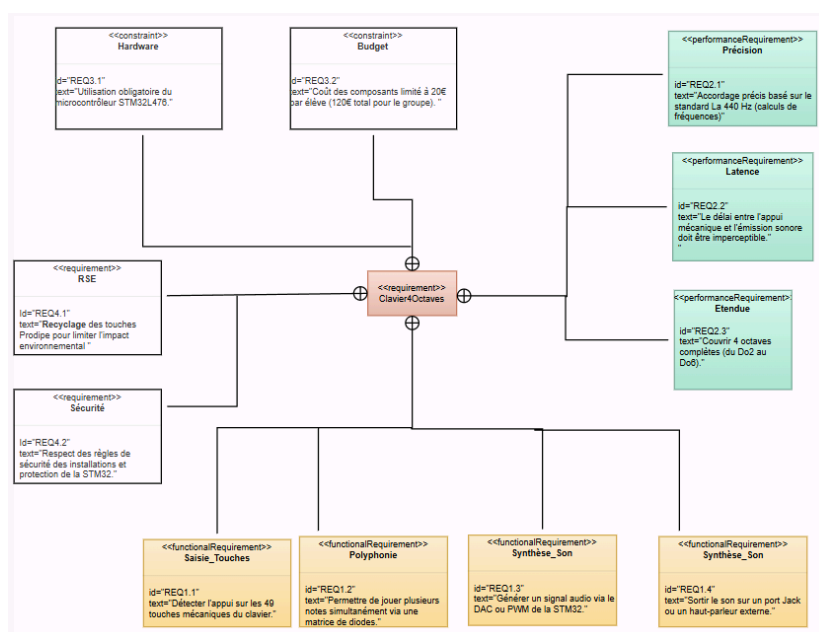


Figure 1 — Diagramme des exigences SysML du projet Clavier4Octaves (réduit à une octave)

Ce diagramme a servi de référence tout au long du projet pour vérifier que nos choix techniques restaient cohérents avec les objectifs initiaux. Il a également été utile lors de la présentation mi-parcours pour montrer l'avancement par rapport aux exigences définies.

Une fois ces exigences posées, nous avons organisé le circuit en quatre blocs fonctionnels distincts, chacun ayant un rôle précis et des interfaces clairement définies avec les autres blocs. Cette décomposition en blocs est représentée dans le schéma d'architecture ci-dessous.

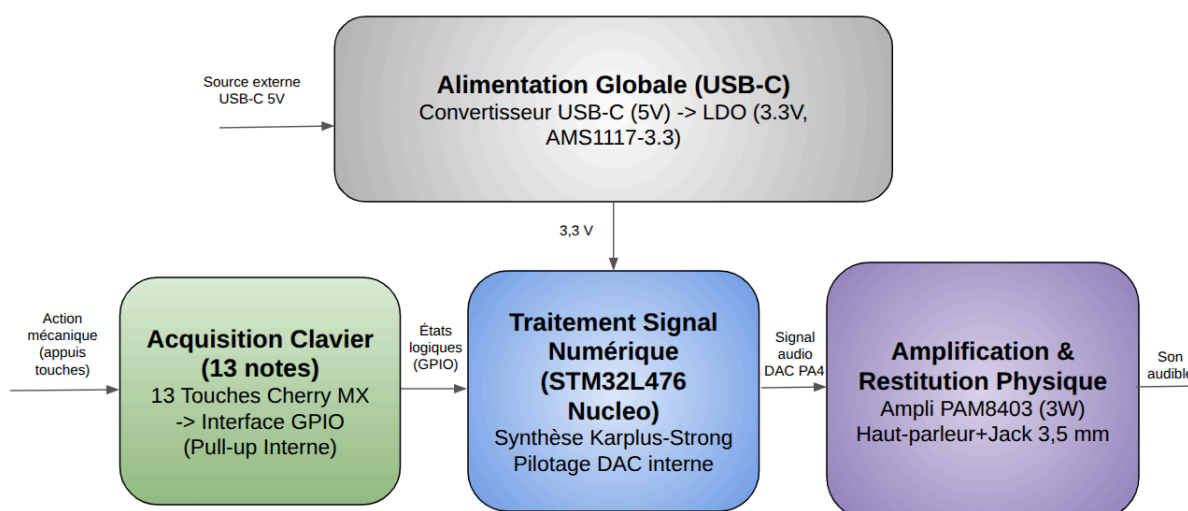


Figure 2 — Architecture fonctionnelle en quatre blocs

I.2 Bloc Acquisition — Détection des touches

Le rôle de ce premier bloc est de convertir l'action mécanique de l'utilisateur (appuyer sur une touche) en un signal électrique interprétable par le microcontrôleur. C'est l'interface directe entre l'instrument et la carte de traitement.

Lors de nos premières réflexions sur ce bloc, deux technologies de capteurs étaient envisageables : les capteurs à effet Hall (magnétiques), qui détectent le mouvement d'un aimant fixé sur la touche, et les switches mécaniques classiques. Les capteurs à effet Hall offrent l'avantage d'une détection analogique (profondeur d'appui), mais leur mise en œuvre est plus complexe, leur coût plus élevé, et surtout ils nécessitent une fixation mécanique personnalisée pour chaque touche. Nous les avons donc écartés.

Notre choix s'est finalement porté sur des switches mécaniques Cherry MX Black Silent. Ces composants présentent d'excellentes caractéristiques techniques pour notre usage : une force d'actuation constante (60cN), un déclenchement très précis, un silence de fonctionnement indispensable pour un instrument audio (la variante Black Silent intégrant des amortisseurs internes) et une durabilité extrême garantie à 50 millions de frappes. Enfin, leur format standardisé a grandement facilité l'intégration sur le schéma KiCad grâce à des empreintes officiellement référencées.

Initialement, le projet prévoyait la récupération et le réemploi des touches d'un piano hors d'usage dans une démarche d'éco-conception. Cependant, suite à des incompatibilités mécaniques avec notre nouveau circuit imprimé, cette approche a été écartée.

Sur le plan électronique : chaque touche relie directement une broche GPIO du STM32 à la masse (GND). Pour garantir un état haut (3,3V) parfaitement stable lorsque la touche est au repos et éviter tout phénomène d'état flottant, chaque ligne matérielle est tirée à l'alimentation par une résistance de tirage (pull-up) externe de 10 kΩ. Lorsque la touche est enfoncée, la broche est court-circuitée à GND et passe à l'état bas (événement détecté par le programme). Ce choix de câblage direct (sans matrice de diodes) est possible car nous disposons d'une broche GPIO dédiée pour chacune des 13 touches, ce qui élimine tout risque de ghosting.

I.3 Bloc Traitement — Carte Nucleo-L476RG

La carte Nucleo-L476RG, embarquant le microcontrôleur STM32L476RGT6, est le cœur du projet. Son utilisation était imposée par le cahier des charges. Elle héberge un processeur ARM Cortex-M4 cadencé à 80 MHz avec unité de calcul en virgule flottante (FPU), 1 Mo de Flash et 128 Ko de SRAM, ce qui est largement suffisant pour notre application de synthèse sonore monodique et polyphonique.

La carte remplit deux missions principales, exécutées en parallèle grâce à l'architecture d'interruptions du STM32. D'une part, elle surveille en permanence l'état des 13 broches GPIO connectées aux switches Cherry MX pour détecter les appuis et relâchements de touches. D'autre part, elle calcule en temps réel, échantillon par échantillon à une fréquence de 44 100 Hz, le signal sonore à produire, et l'envoie au convertisseur numérique-analogique (DAC) intégré sur la broche PA4.

Nous avons choisi le DAC intégré plutôt qu'une sortie PWM : le DAC produit directement un signal analogique propre, sans harmoniques parasites liées à la modulation de largeur d'impulsion. Un signal PWM nécessiterait un filtre passe-bas externe pour reconstituer le signal audio, ajoutant des composants et une potentielle dégradation de la qualité sonore. La résolution de 12 bits du DAC du STM32L476 offre une plage dynamique de 72 dB.

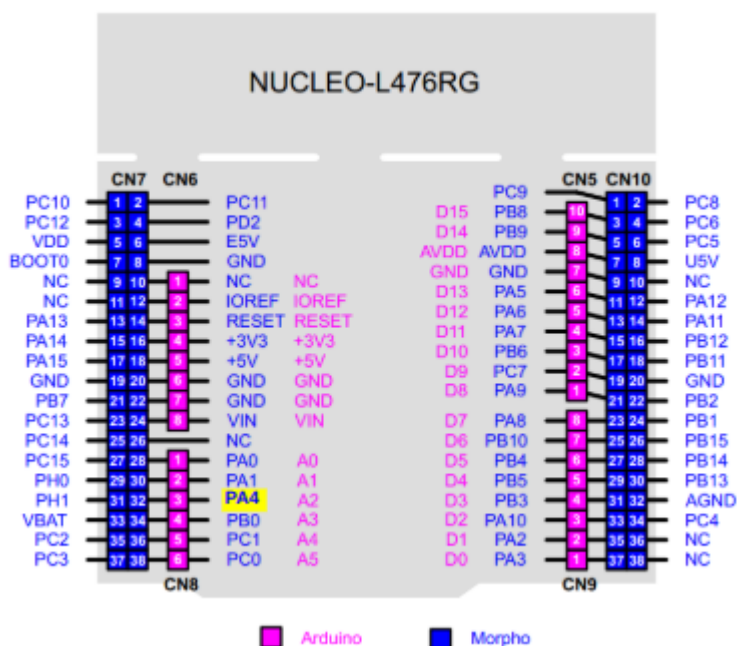


Figure 3 — Brochage de la carte Nucleo-L476RG et localisation de la sortie analogique DAC (PA4)

I.4 Bloc Alimentation — USB-C et régulation 3,3V

L'alimentation de notre système devait être à la fois pratique, sûre et économique. Nous avons écarté la batterie 9V : elle est trop encombrante et peu fiable pour des sessions prolongées. Nous nous sommes alors tournés vers l'alimentation par connecteur USB-C.

Contrairement aux anciennes normes, un chargeur USB-C moderne (Power Delivery) ne délivre pas de courant aveuglément : il attend que l'appareil branché s'identifie via deux broches spécifiques (CC1 et CC2).

Pour assurer le bon fonctionnement de notre piano, nous avons ajouté deux résistances de 5,1 k Ω sur ces broches. Ce choix de conception a un double objectif :

- Activer l'alimentation : Sans ces résistances, le chargeur ne détecterait aucun appareil et refuserait de fournir du courant.
- Garantir la sécurité matérielle : Cette valeur de 5,1 k Ω indique au chargeur que notre circuit est un récepteur standard de 5 V. Cela empêche le déclenchement d'une charge rapide (9V ou 12V) qui enverrait une tension destructrice pour nos composants.

Une fois la tension de 5V sécurisée via un interrupteur unipolaire (SW_SPST) permettant d'allumer l'instrument, l'énergie est séparée en deux différentes lignes d'alimentation.

D'une part, la ligne 5V alimente directement l'amplificateur audio PAM8403. Cela permet de fournir la puissance nécessaire aux pics de consommation (c'est-à-dire quand un composant a besoin d'une très grande quantité d'énergie d'un seul coup, notamment lors des sons graves) sans risquer de perturber le reste du circuit.

D'autre part, cette tension de 5V est abaissée à 3,3V par le régulateur linéaire AMS1117-3.3 (monté en boîtier SOT-223) pour alimenter la carte Nucleo et la détection des switches. Ce régulateur, capable de fournir jusqu'à 800 mA, est ainsi largement dimensionné pour le cerveau numérique. Des condensateurs de filtrage sont placés en entrée (10 μ F) et en sortie (22 μ F) du régulateur pour lisser cette tension de 3,3V et garantir un fonctionnement sans plantage du microcontrôleur.

I.5 Bloc Sortie Audio — Amplification et sorties

Le signal produit par le DAC de la Nucleo est un signal analogique de faible niveau (quelques centaines de millivolts d'amplitude). Ce signal est insuffisant pour alimenter directement un haut-parleur, qui nécessite une puissance de l'ordre du watt pour produire un volume sonore audible. Un étage d'amplification est donc indispensable.

Nous avons sélectionné le module PAM8403, un amplificateur audio de Classe D capable de fournir 2 $\times$ 3 W en sortie. Son excellent rendement énergétique (jusqu'à 90 %) minimise la dissipation de chaleur. Cet amplificateur intègre également une protection contre les courts-circuits en sortie et contre les surtensions, ce qui le rend fiable pour notre usage.

Le signal audio du DAC passe d'abord par un potentiomètre double de 10 k Ω qui joue le rôle de contrôle de volume : en tournant ce bouton, l'utilisateur fait varier la résistance, atténuant plus ou moins le signal avant qu'il n'atteigne l'entrée de l'amplificateur.

En sortie d'amplificateur, le signal est envoyé par défaut vers le haut-parleur de 3 W intégré. Pour offrir plus de flexibilité, nous avons opté pour une prise Jack 3,5 mm à coupure matérielle (Switched Jack) : l'insertion d'un casque déconnecte ainsi automatiquement et physiquement le haut-parleur. Enfin, pour garantir une écoute sans bruit parasite, des condensateurs de couplage (bloquant la composante continue) et de découplage d'alimentation ont été placés stratégiquement au plus près de l'amplificateur afin de lisser le signal.

I.6 Tableau récapitulatif des composants

Le tableau suivant récapitule l'ensemble des composants retenus pour chacun des quatre blocs fonctionnels, avec leurs références fabricant, fournisseurs et coûts unitaires :

Bloc	Composant	Référence fabricant	Fournisseur	Prix unit. (€)
Acquisition	Switch mécanique (×13)	Cherry MX1A-11NN	Mouser	1,10
Traitement	Carte Nucleo-L476RG	Nucleo-L476RG	École	—
Alimentation	Connecteur USB-C	USB4105-GF-A 3588205	Farnell	0,90
Alimentation	Régulateur AMS1117-3.3	AMS1117-3.3	École	—
Alimentation	Interrupteur	R1973ABLKILGF3	Digikey	2,40
Sortie	Amplificateur PAM8403	3373823	Farnell	1,40
Sortie	Haut-parleur 3 W	1528-2752-ND	Digikey	4,20
Sortie	Prise Jack 3,5 mm	FC681375V	Digikey	1,90
Sortie	Potentiomètre 10 kΩ	RK16312A0BKR	Mouser	6,80

Tableau 1 — Liste complète des composants par bloc fonctionnel avec fournisseurs et prix

II. Réalisation Pratique

II.1 Programmation

Avant de commencer la programmation en langage C sur la carte Nucleo, nous avons cherché à valider notre méthode de génération sonore sur ordinateur via un prototype Python. L'objectif n'était pas d'utiliser des sons préenregistrés, mais de générer le signal audio mathématiquement en temps réel.

Pour ce faire, nous devions choisir une méthode de synthèse sonore adaptée aux ressources limitées de notre microcontrôleur. Plusieurs options ont été écartées :

- La synthèse additive (additionner plusieurs ondes sinusoïdales) exige un temps de calcul trop long et trop d'opérations par échantillon.
- La synthèse FM (modulation de fréquence) est particulièrement complexe à implémenter de zéro.

Notre choix s'est porté sur l'algorithme de Karplus-Strong, étant facile à implémenter et rapide à exécuter. Cet algorithme permet de reproduire mathématiquement le comportement physique d'une corde frappée. Il se divise en trois étapes :

1. L'impact initial (l'excitation) : Lorsqu'on appuie sur une touche de piano, un marteau vient frapper brutalement la corde. Dans notre programme, nous simulons cet impact en générant une très courte liste de nombres aléatoires (ce que l'on appelle un « bruit blanc », avec des valeurs comprises entre -1 et +1). À l'écoute, ce signal brut se traduit par une impulsion sèche et très brève. Il agit comme l'excitation mécanique initiale de notre corde virtuelle, déclenchant ainsi la note.

2. La note jouée (la résonance) : Une fois frappée, l'onde mécanique fait des allers-retours le long de la corde. Pour simuler cela, notre programme va lire en boucle la liste de nombres générée à l'étape précédente. La taille de cette liste définit la note que l'on va entendre. Pour bien le visualiser, on peut imaginer le manche d'une guitare : si l'on pose un doigt sur une case, on raccourcit physiquement la corde. L'onde fait ses allers-retours beaucoup plus vite, et le son devient aigu. Dans notre code, le principe est le même : si notre liste de nombres est petite, elle se répète très vite et crée une note aiguë. Si elle est grande, elle met du temps à se répéter et crée une note grave. Par exemple, pour jouer un La à 440 Hz (sachant que notre système lit 44 100 valeurs par seconde), un petit calcul nous indique que notre liste doit contenir exactement 100 valeurs ($44\ 100 / 440 \approx 100$).

3. L'étouffement du son (l'amortissement) : Dans la réalité, l'énergie d'une corde qui vibre finit par se dissiper dans l'air, et le son s'éteint doucement. Pour recréer cet adoucissement, notre programme ne se contente pas de répéter bêtement les nombres en boucle. À chaque passage, il calcule la moyenne mathématique entre les valeurs voisines. Faire cette moyenne agit comme un filtre passe-bas qui « arrondit les angles » de notre signal. Au fur et à mesure, le bruit chaotique du départ se transforme en une belle onde musicale, filtrant les hautes fréquences, et son volume diminue progressivement jusqu'au silence total.

Pour automatiser la vitesse de cet étouffement en fonction de la durée de note souhaitée, nous avons développé une formule mathématique basée sur la fonction Arctangente : $\text{damping} = (2 * \arctan(140 * \text{duree})) / \pi$. Cette formule garantit que le paramètre d'amortissement reste strictement compris entre 0 et 1. Elle permet une saturation naturelle

évitant toute divergence du signal. Nous avons également ajouté une seconde de marge à la fin de chaque note pour permettre à l'onde de s'éteindre jusqu'au zéro absolu, sans coupure brusque qui produirait un « clic » audio désagréable.

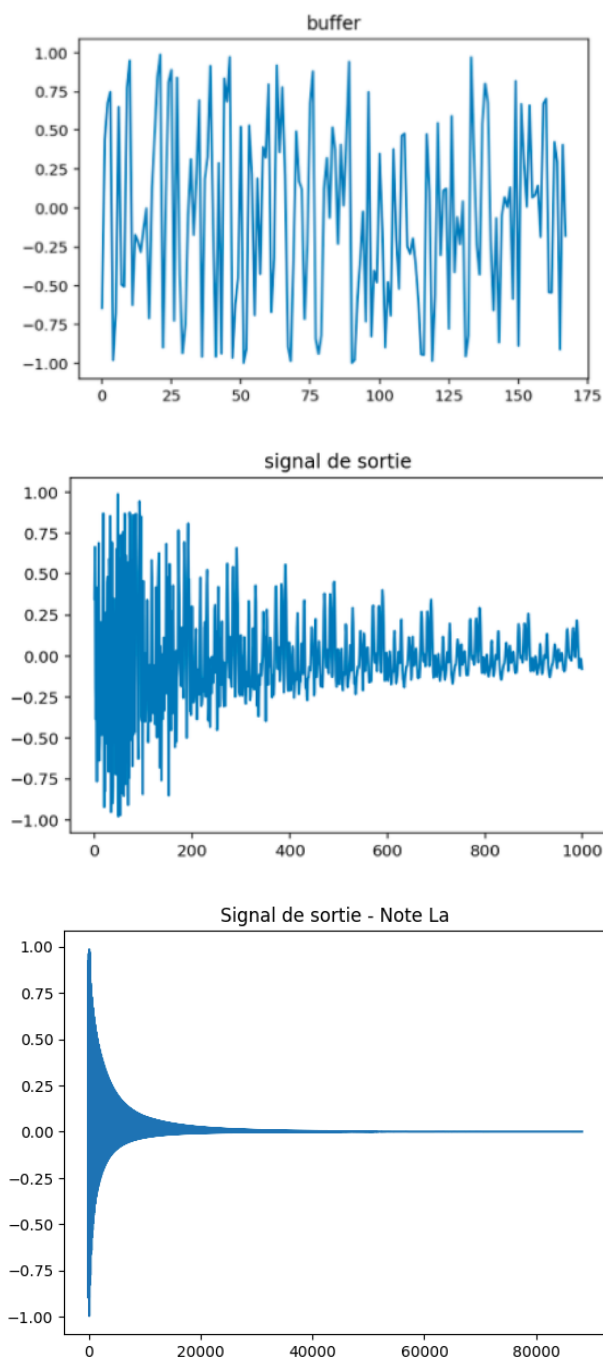


Figure 4 — Évolution du signal Karplus-Strong au fil des itérations

II.1.2 Configuration des périphériques (STM32CubeMX)

Avant d'écrire le code applicatif, nous avons utilisé l'outil graphique STM32CubeMX pour configurer les ressources matérielles du microcontrôleur. Cette étape consiste à attribuer un rôle physique à chaque broche (pin) de la carte et à paramétrer les périphériques internes. STM32CubeMX génère ensuite automatiquement les fonctions d'initialisation correspondantes, ce qui nous a permis de nous concentrer sur l'écriture de l'algorithme musical plutôt que sur la configuration bas niveau des registres.

Trois types de ressources ont été configurés. D'une part, le DAC sur la broche PA4 : le convertisseur numérique-analogique est activé sur ce canal et configuré pour une résolution de 12 bits, ce qui donne 4096 niveaux de tension entre 0 et 3,3 V (suffisant pour une qualité audio satisfaisante). D'autre part, les broches GPIO en entrée : chacune des 13 broches reliées aux switches Cherry MX est configurée en mode INPUT avec résistance de tirage pull-up interne activée, garantissant une lecture stable (état haut au repos, état bas lors de l'appui). Enfin, le Timer TIM4 : configuré pour déclencher une interruption à exactement 44 100 Hz (période de 22,7 μ s), il sert de métronome électronique pour la génération des échantillons audio.

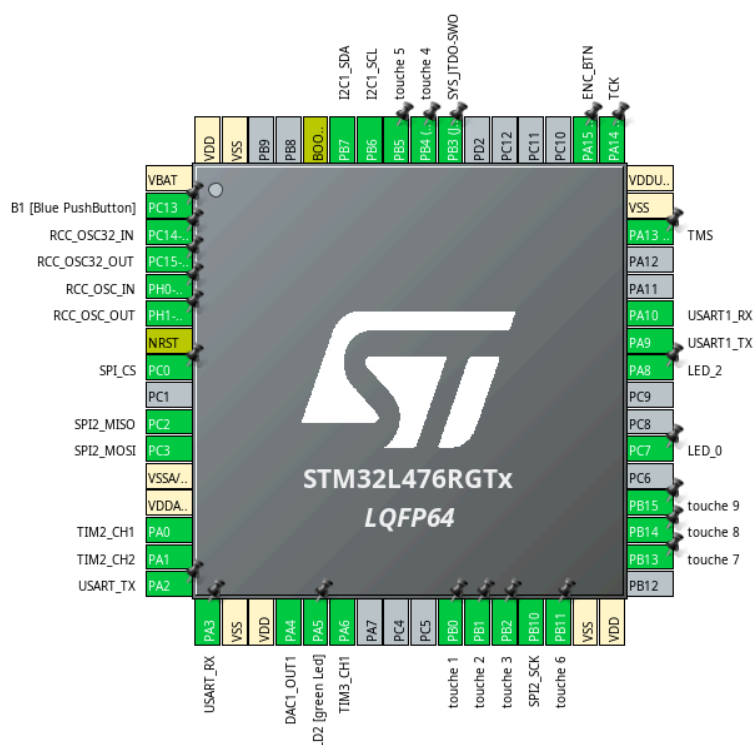


Figure 5 — Configuration des broches dans STM32CubeMX

II.1.3 Bibliothèque Karplus-Strong en C (karplus_strong.c / .h)

Sur le microcontrôleur STM32, la génération doit être continue et effectuée en temps réel : la carte doit fournir un nouvel échantillon toutes les 22,7 μ s, sans interruption.

Pour structurer ce programme, nous avons créé une bibliothèque dédiée encapsulant toutes les variables de la corde virtuelle dans une structure C personnalisée KarplusStrong_t (taille du buffer, facteur d'amortissement, tableau de valeurs, index courant). La bibliothèque s'articule autour de deux fonctions :

- KS_Init() : initialise la structure avec la fréquence et l'amortissement souhaités. Elle calcule la taille N du tableau (SAMPLE_RATE / frequency) et intègre des sécurités matérielles (if)

pour empêcher cette taille de dépasser la mémoire disponible (MAX_BUFFER_SIZE). Le tableau est ensuite rempli de bruit blanc généré par la fonction rand(), ramené mathématiquement à une échelle de valeurs flottantes comprises entre -1,0 et +1,0.

- KS_Step() : appelée 44 100 fois par seconde par l'interruption du Timer, cette fonction lit la valeur courante, calcule la moyenne avec la valeur suivante, et multiplie le résultat par le facteur d'amortissement (damping). Elle écrase ensuite l'ancienne valeur et avance l'index d'un cran (avec un retour à 0 via l'opérateur modulo %, créant ainsi le comportement d'un buffer circulaire). Elle retourne enfin la valeur calculée, qui sera convertie dans le programme principal en entier 12 bits pour être envoyée au DAC.

L'utilisation du buffer circulaire constitue la principale optimisation par rapport au prototype Python : au lieu de décaler physiquement toutes les valeurs du tableau à chaque itération (complexité $O(N)$), le programme fait simplement avancer un index (opération en $O(1)$). Sur un microcontrôleur cadencé à 80 MHz ne disposant que de 22,7 μ s entre deux calculs, cette optimisation d'accès mémoire était essentielle.

```

17 void KS_Init(KarplusStrong_t *ks, float frequency, float damping)
18 {
19     uint16_t N = (uint16_t)(SAMPLE_RATE / frequency);
20     if (N > MAX_BUFFER_SIZE) N = MAX_BUFFER_SIZE;
21     if (N < 2) N = 2; // sécurité
22
23     ks->size = N;
24     ks->index = 0;
25     ks->damping = damping;
26
27     // Bruit blanc entre -1.0 et 1.0
28     for (uint16_t i = 0; i < N; i++) {
29         ks->buffer[i] = ((float)rand() / (float)RAND_MAX) * 2.0f - 1.0f;
30     }
31 }
32
33 float KS_Step(KarplusStrong_t *ks)
34 {
35     float current = ks->buffer[ks->index];
36     uint16_t next_index = (ks->index + 1) % ks->size;
37
38     float avg = ks->damping * 0.5f *
39         (ks->buffer[ks->index] + ks->buffer[next_index]);
40
41     ks->buffer[ks->index] = avg;
42     ks->index = next_index;
43
44     return current;
45 }

```

Figure 6 — Bibliothèque karplus_strong.c (extrait des deux fonctions principales)

II.1.4 Implémentation logicielle et simulation audio

Afin de valider notre algorithme de génération sonore, nous avons développé un programme de test en langage C, conçu pour être exécuté directement sur ordinateur.

Cette implémentation a permis de tester et de valider trois concepts avancés de notre synthèse sonore :

- L'enveloppe sonore (ADSR) : L'algorithme a été enrichi d'une enveloppe temporelle (Attack, Decay, Sustain, Release). Cela permet de sculpter le volume de la note pour qu'elle ne commence ni ne s'arrête de manière brutale, rendant la frappe beaucoup plus réaliste et naturelle à l'oreille.

- La gestion de la polyphonie : Le programme a été structuré pour instancier plusieurs algorithmes de Karplus-Strong en parallèle, ce qui permet de jouer plusieurs notes simultanément. Les tests ont notamment permis de générer un accord complet de Do majeur (associant les fréquences 261,63 Hz, 329,63 Hz et 392,00 Hz) pour vérifier que les signaux s'additionnaient correctement.

- La génération de fichiers audio : Plutôt que d'envoyer le son vers le convertisseur de la carte électronique, ce programme de test stocke les millions de valeurs mathématiques calculées et les exporte sous la forme d'un fichier audio standard (format .wav).

Cette étape de simulation nous a permis d'écouter les échantillons produits.

```
for (int n = 0; n < nb_notes; n++) {  
    float s = KS_Step(&ks_accord[n]) * ADSR_Step(&envs[n]);  
    mix += s;  
}  
  
mix /= nb_notes; // normalisation  
buffer_f[pos++] = mix;
```

Figure 7 — Extrait du code de simulation sur PC (application de l'enveloppe ADSR et mixage pour la polyphonie)

II.2 Conception électronique sur KiCad

II.2.1 Architecture du schéma électrique

La conception matérielle a débuté par la réalisation du schéma électrique sous KiCad. L'objectif était de concevoir une carte d'extension (shield) venant se connecter par-dessus la carte Nucleo-L476RG, exploitant ses broches via deux rangées de connecteurs headers. Cette approche modulaire permet de ne pas souder le microcontrôleur sur notre PCB, préservant ainsi la Nucleo pour des futures réutilisations.

Pour améliorer la lisibilité du schéma (c'est-à-dire pour éviter un mélange désordonné de fils traversant toute la feuille), nous avons utilisé des labels de nets nommés (AUDIO_OUT, Note_1 à Note_13, +3.3V, GND, PWR_FLAG). KiCad relie automatiquement toutes les broches partageant le même label, ce qui simplifie la vérification ERC (Electrical Rules Check) et facilite la modification ultérieure du schéma. Le schéma est organisé en quatre zones distinctes correspondant aux quatre blocs fonctionnels : alimentation (haut gauche), interface Nucleo (haut droite), amplification audio (centre), et switches mécaniques (bas).

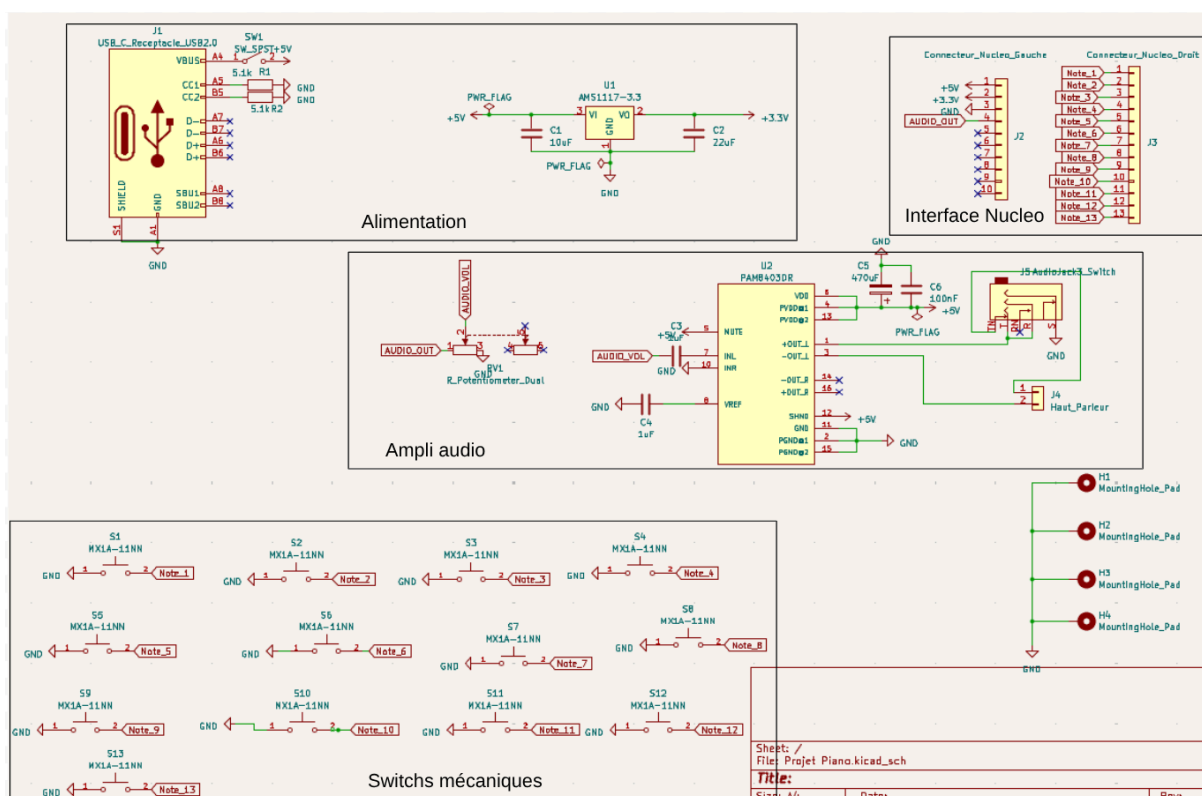


Figure 8 — Vue d'ensemble du schéma électrique sous KiCad

Les figures suivantes présentent en détail chaque sous-schéma.

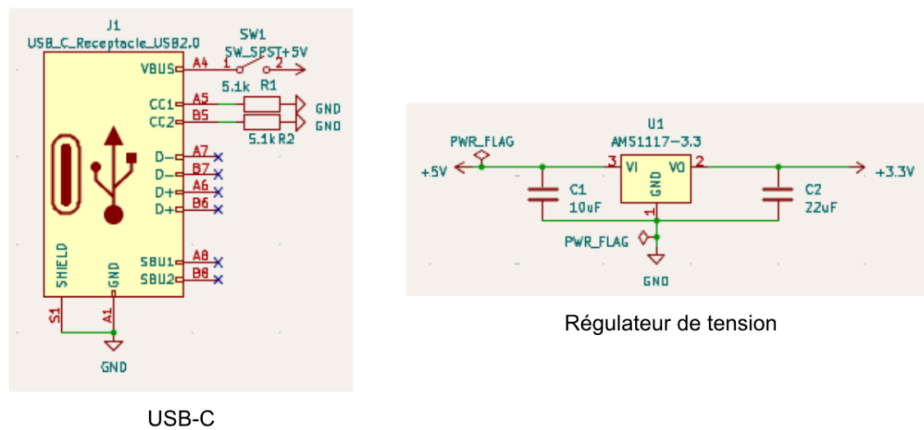


Figure 9 — Bloc alimentation : connecteur USB-C et régulateur AMS1117-3.3

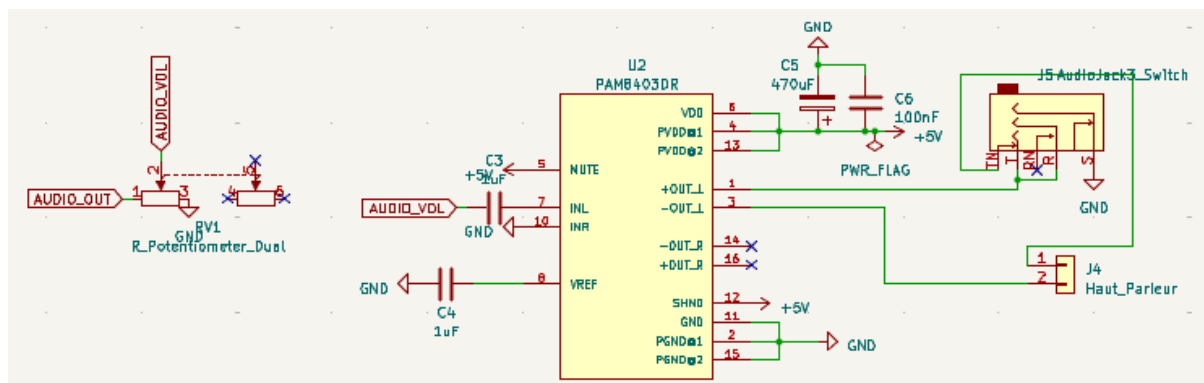


Figure 10 — Bloc sortie audio : amplificateur PAM8403 et sorties

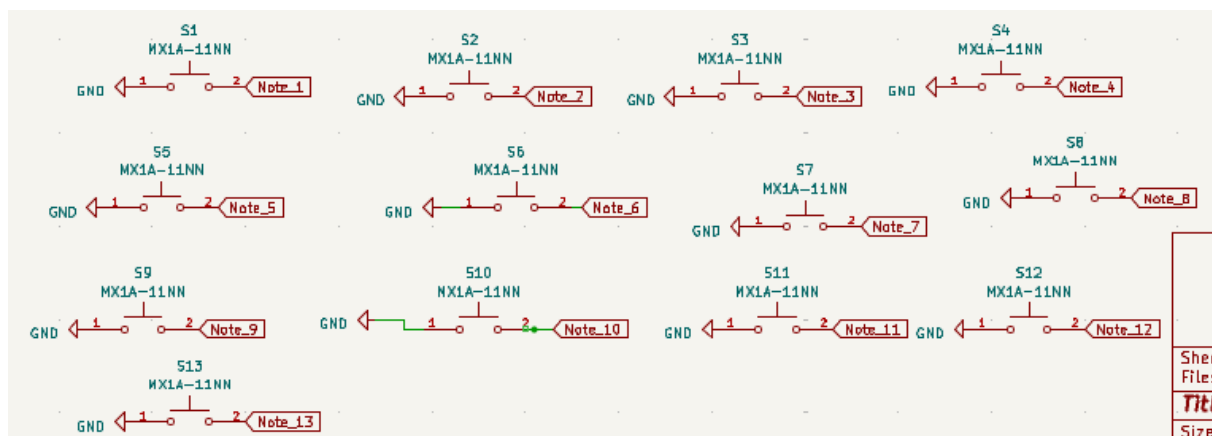


Figure 11 — Bloc acquisition : les 13 switchs Cherry MX

II.2.2 Sélection des empreintes (footprints)

Le passage du schéma théorique au circuit imprimé nécessite d'associer à chaque symbole une empreinte physique (footprint), c'est-à-dire la représentation exacte de l'espace que le composant occupera sur le PCB, avec les pastilles de soudure aux dimensions correctes et les trous de perçage si nécessaire.

Pour les switches Cherry MX, nous avons utilisé l'empreinte officielle MX1A-11NN, qui inclut deux pastilles pour les broches électriques, deux perçages pour les plots de fixation mécanique en plastique (non connectés électriquement), et un repère de polarité. Pour l'amplificateur audio PAM8403, nous avons utilisé l'empreinte CMS de type SO-16 (Composant Monté en Surface), adaptée au boîtier compact de la puce sélectionnée. Le régulateur de tension AMS1117 est également un composant CMS (boîtier SOT-223), dont la particularité est de posséder une large languette centrale qui sert à la fois de connexion électrique et de dissipateur thermique. Enfin, le connecteur d'alimentation USB-C nécessite une empreinte hybride technique : ses broches de signal sont soudées en surface, tandis que son bouclier métallique traverse la carte de part en part (THT) pour assurer un ancrage mécanique solide, indispensable face aux branchements et débranchements répétés de l'utilisateur.

Réf.	Désignation	Référence	Format	Qté
S1–S1 3	Switchs Cherry MX	MX1A-11NN	THT	13
U2	Amplificateur audio 3 W	PAM8403DR	CMS (SO-16)	1
U1	Régulateur de tension 3,3 V	AMS1117-3.3	CMS (SOT-223)	1
J1	Connecteur USB-C	USB-C Receptacle USB2.0	CMS/Hybride	1
J5	Prise Jack 3,5 mm avec coupure	AudioJack3_Switch	THT	1
RV1	Potentiomètre volume double	Potentiomètre Dual 10 kΩ	THT	1
SW1	Interrupteur marche/arrêt	SW_SPST	THT	1
R1, R2	Résistances d'activation USB-C (5,1 kΩ)	Résistance axiale	THT	2
C1, C2	Condensateurs filtrage régulateur (10 µF, 22 µF)	Électrolytique	THT	2
C3–C6	Condensateurs filtrage ampli (1 µF (x2), 470 µF, 100 nF)	Électrolytique/cérami que	THT/CMS	4
J2, J3	Connecteurs interface Nucleo (headers 2×10)	Conn_01x10 Conn_01x13	THT	2
J4	Connecteur haut-parleur	Conn_01x02	THT	1

Tableau 2 — Composants, empreintes KiCad et caractéristiques de montage

II.2.3 Placement des composants et ergonomie

Le placement des composants sur le PCB a été pensé en priorité pour l'ergonomie de l'utilisateur. Après avoir ouvert le fichier `.kicad_pcb` et défini le contour de la carte pour qu'elle s'emboîte au-dessus de la Nucleo, nous avons divisé la surface en deux zones fonctionnelles.

La zone de jeu (moitié inférieure de la carte) accueille les 13 empreintes Cherry MX, disposées pour reproduire fidèlement la disposition d'une octave de piano : les touches blanches (Do, Ré, Mi, Fa, Sol, La, Si, Do) sont alignées horizontalement en bas, et les touches noires (Do#, Ré#, Fa#, Sol#, La#) sont décalées vers le haut et centrées entre les touches blanches correspondantes. L'entraxe entre les touches a été calculé pour qu'un joueur de taille adulte puisse appuyer sur chaque touche sans gêne.

La zone technique (moitié supérieure de la carte) regroupe tous les composants électroniques : le bloc alimentation (connecteur USB-C, interrupteur, régulateur AMS1117 et ses condensateurs) sur le bord supérieur gauche, le bloc audio (amplificateur PAM8403, potentiomètre, prise Jack, connecteur haut-parleur) sur le bord supérieur droit, et les connecteurs d'interface avec la Nucleo sur le bord droit.

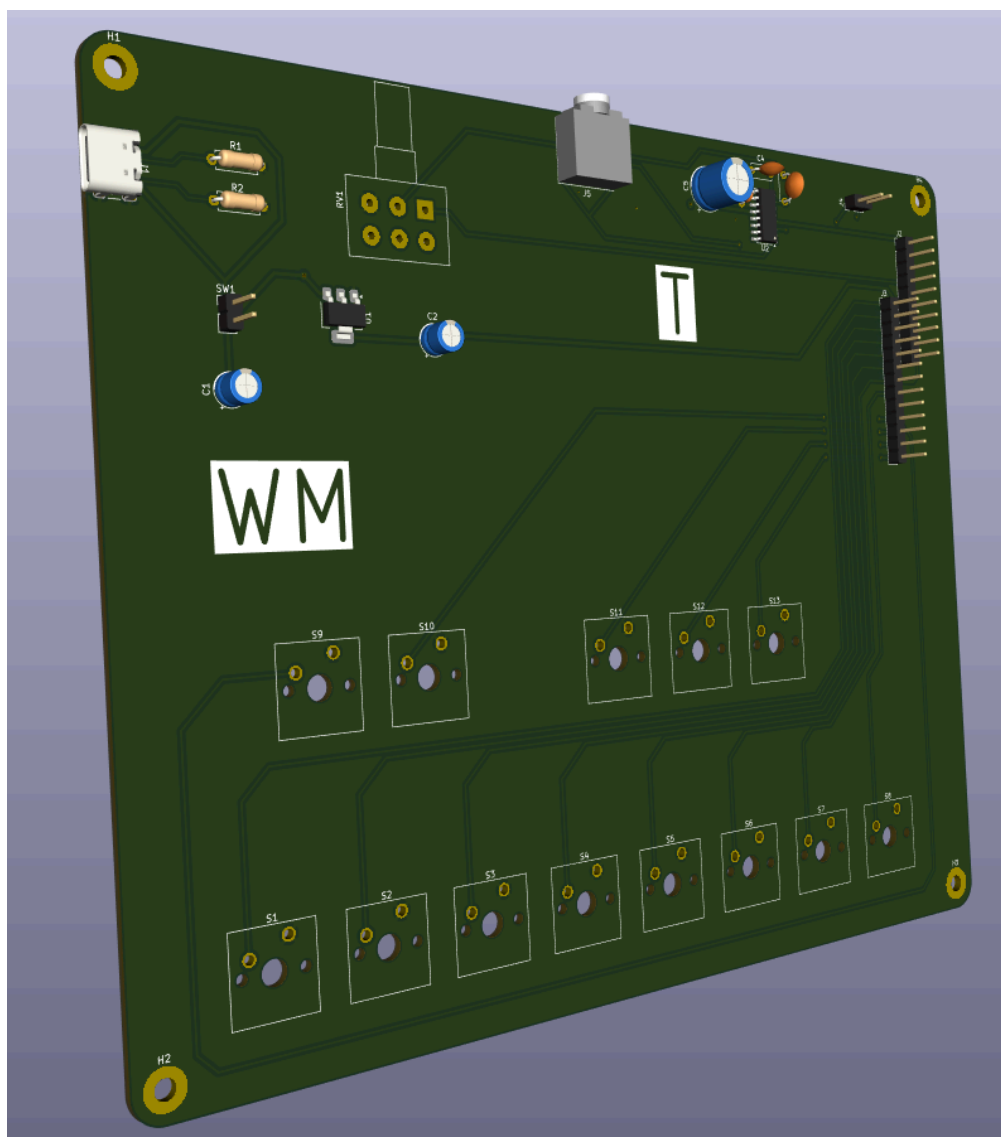


Figure 12 — Vue 3D du PCB (KiCad) - disposition des composants

II.2.4 Routage du circuit imprimé et plan de masse

Le routage a été réalisé sur deux couches de cuivre (face TOP en rouge, face BOTTOM en bleu) en respectant strictement les règles de conception (DRC) : largeur de piste et isolation minimales fixées à 0,3 mm (avec un standard de 0,4 mm privilégié dès que l'espace le permettait), et diamètres de perçage ajustés selon les pas réglementaires de l'atelier.

La stratégie de routage a suivi plusieurs principes. Les longues pistes reliant les 13 switches aux connecteurs de la Nucleo (signaux numériques GPIO) ont été acheminées sur la couche BOTTOM pour éviter de traverser la zone d'amplification audio, où elles pourraient induire des parasites sur le signal analogique sensible. Les pistes d'alimentation (+5V, +3.3V) ont été tracées avec une largeur augmentée afin de supporter les courants plus élevés sans chute de tension significative.

Un plan de masse GND a été appliqué en remplissage sur les deux faces de la carte (zone fill). Ce plan remplit tous les espaces vides non occupés par des pistes ou des composants, et est connecté au potentiel GND. Il constitue un retour de courant de faible impédance pour tous les composants, réduisant les chutes de tension parasites, et il forme un bouclier électromagnétique protégeant le signal audio des interférences extérieures. L'absence d'un tel plan de masse se traduirait typiquement par un bruit de fond perceptible dans le haut-parleur (souffle ou grésillements synchrones avec la lecture des touches).

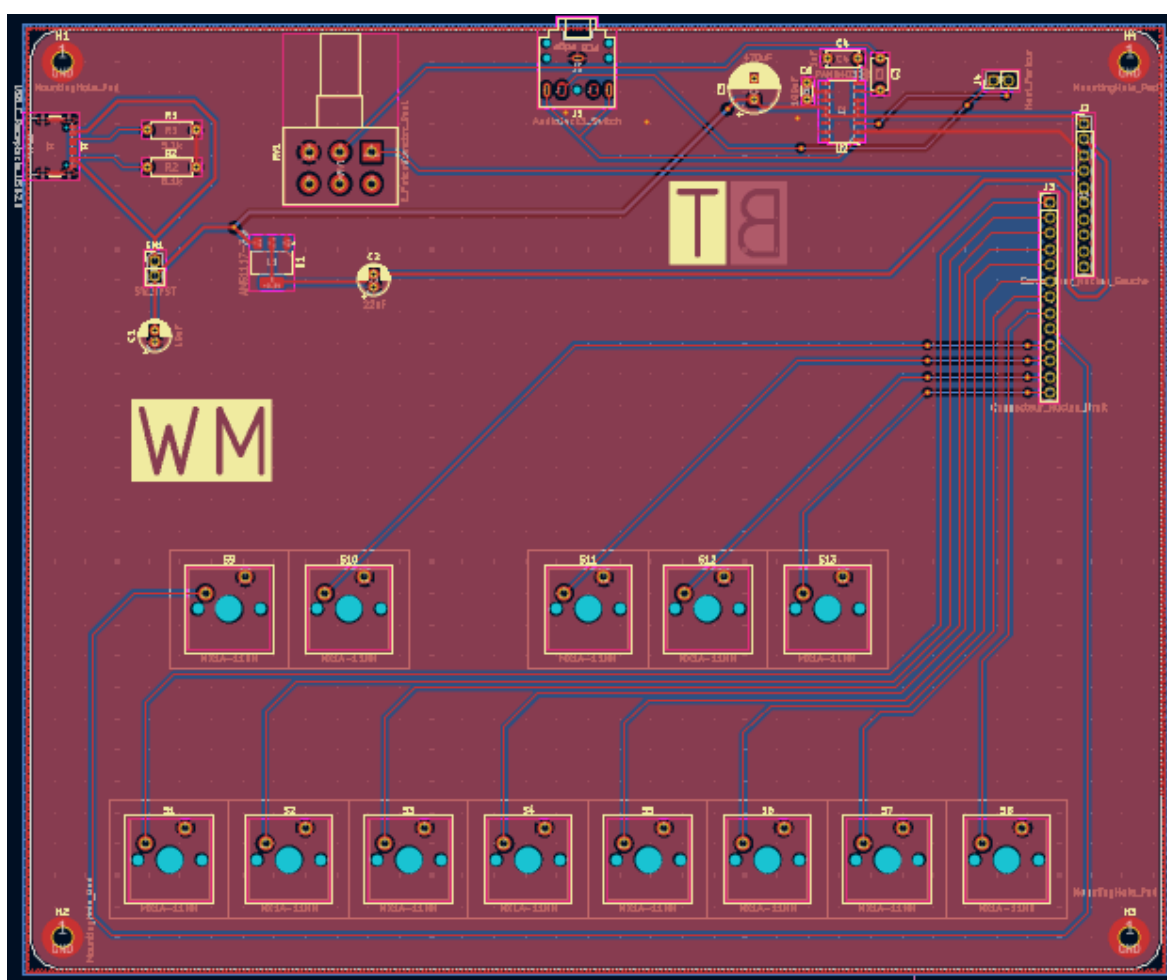


Figure 13 — Routage du circuit imprimé : couche TOP (rouge) et couche BOTTOM (bleu)

II.3 Soudage et Assemblage

L'assemblage de la carte s'est déroulé en plusieurs étapes

-Soudage au four : Widad et Julien ont appliqué de la pâte à braser sur le circuit imprimé à l'aide d'un pochoir (stencil), puis ont passé la carte au four pour souder les premiers composants.

-Soudage manuel : Pendant ce temps, Wissem a rassemblé le reste des composants au FabLab. L'équipe a ensuite soudé ces éléments manuellement à l'aide de fers à souder.

-Finitions (hors séance) : intégration du port Jack et du haut-parleur et constatation de l'oubli des empreintes pour les résistances de tirage (pull-up) sur le circuit imprimé. Les 13 résistances de 10 k Ω ont alors été soudées manuellement au dos de la carte.

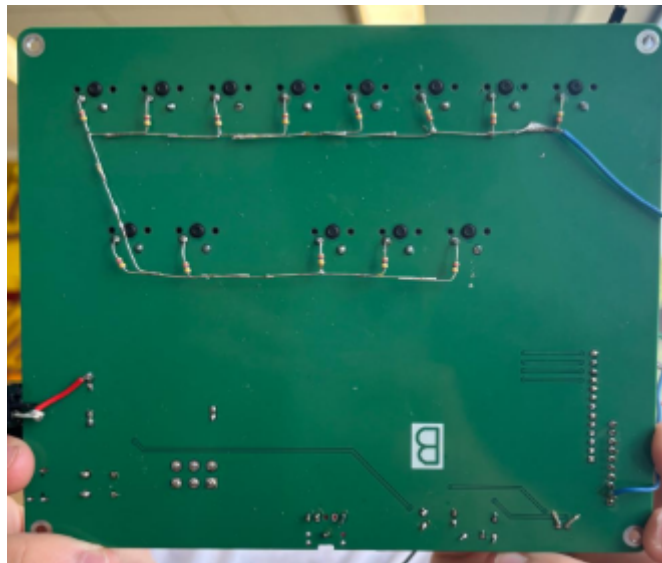


Figure 14 — Ajout manuel des 13 résistances de tirage (pull-up) au dos du circuit imprimé

Enfin, pour protéger les soudures et les composants situés au dos de la carte contre les courts-circuits, un socle de protection a été modélisé et fabriqué en impression 3D pour y reposer le circuit imprimé.

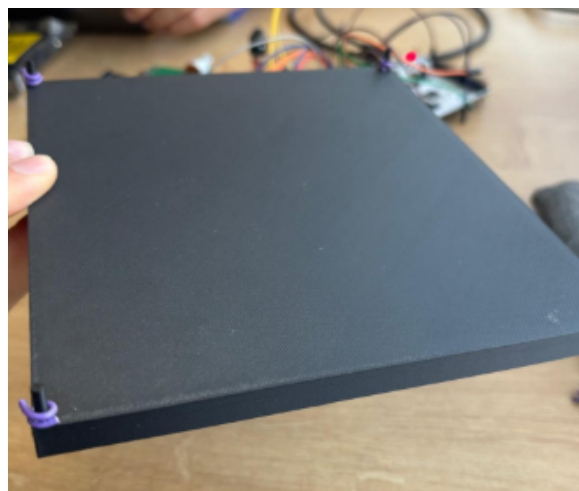


Figure 15 — Support mécanique du clavier fabriqué par impression 3D

III. Gestion de Projet, RSE et Sécurité

III.1 Méthodologie et Répartition des rôles

Le tableau suivant présente la répartition des rôles que nous nous sommes attribués lors de la première séance.

Rôle	Responsable
Répartition des tâches	Léo ROUPENEL
Respect des délais	Julien FAUVEL
Commandes	Widad NORI
Tests	Wisseem M'TIR
Rapport hebdo + site web	Élie CORBIÈRE
Rapport final + diapo	Claire MESTRE PINCHO

Tableau 3 — Répartition des rôles dans le groupe

En pratique, notre organisation a évolué de manière fluide et collaborative. Les choix techniques globaux ont fait l'objet de discussions communes. Pour la réalisation technique, des sous-groupes se sont formés :

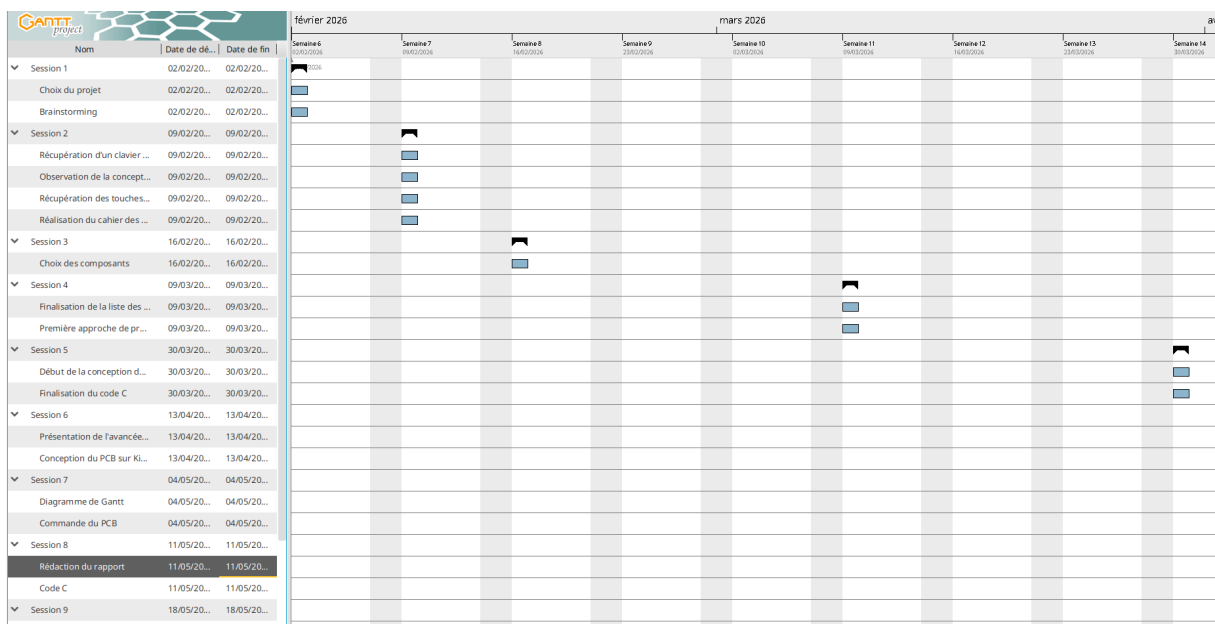
Tâche	Membres impliqués
Conception du circuit sous KiCad	Léo, Élie et Wisseem
Sélection et commande des composants	Élie, Widad, Claire, Léo
Soudage	Widad, Wisseem, Léo, Julien
Programmation	Julien, Wisseem, Widad
Site web	Élie
Rapport final + diapo	Claire

Tableau 4 — Répartition effective des tâches au cours du projet

Chronologie et diagramme de Gantt

Sess ion	Date	Travaux réalisés
S1	02/02/2026	Choix du projet, brainstorming sur les contraintes et les approches possibles
S2	09/02/2026	Récupération d'un clavier non fonctionnel, rédaction du cahier des charges, diagramme SysML
S3	16/02/2026	Sélection et comparaison des composants, première liste BOM
S4	09/03/2026	Finalisation de la nomenclature et passation des commandes, prototype Python de l'algorithme Karplus-Strong (validation du modèle mathématique)
S5	30/03/2026	Début de la conception PCB sous KiCad (schéma électrique), premiers essais du code C
S6	13/04/2026	Présentation mi-parcours, début du routage PCB
S7	04/05/2026	Finalisation du routage PCB (et commande), création du diagramme de Gantt
S8	11/05/2026	Rédaction du rapport, code C
S9	18/05/2026	Réception du PCB, soudage et assemblage du PCB, finalisation du site internet, rédaction du rapport
S10	28/05/2026	Soutenance

Tableau 5 — Chronologie détaillée du projet session par session



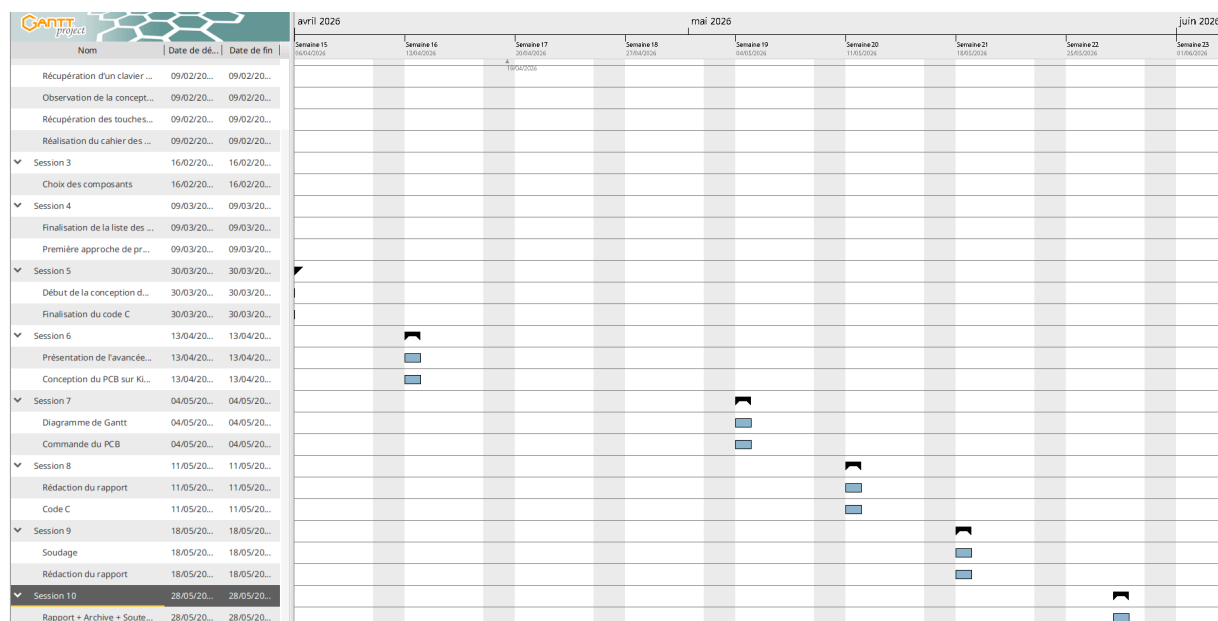


Figure 16 — Diagramme de Gantt du projet (fév. – mai 2026)

III.2 Démarche RSE (Responsabilité Sociétale des Entreprises)

La conception de notre piano numérique a intégré dès le départ une réflexion sur l'impact environnemental, structurée autour de trois choix techniques concrets :

- Lutte contre l'obsolescence et réparabilité : nous avons investi dans des switches mécaniques Cherry MX, reconnus pour leur longue durée de vie (50 millions de frappes). De plus, ces touches et les interfaces subissant le plus de contraintes (prise Jack, potentiomètre, interrupteur) ont été choisies en format traversant (THT). Cela permet de dessouder et de remplacer une pièce défectueuse simplement avec un fer à souder.
- Architecture modulaire : En concevant notre circuit imprimé sous forme de shield (carte d'extension emboîtable) plutôt que d'y souder directement un microcontrôleur, nous préservons la carte Nucleo-L476RG. Ce composant, qui est le plus complexe et coûteux du système, n'est pas immobilisé et pourra être débranché pour servir à l'avenir.

III.3 Analyse des risques et Sécurité

La conception du clavier a intégré plusieurs mesures de prévention pour protéger l'utilisateur final et pour garantir l'intégrité du matériel :

- Sécurité électrique de l'utilisateur (TBTS) : L'ensemble du circuit fonctionne sous une Très Basse Tension de Sécurité (5 V) fournie par le port USB-C. Le risque d'électrocution est donc nul en conditions normales, l'isolation avec le secteur 230 V étant assurée en amont par le chargeur externe de l'utilisateur.
- Protection matérielle USB-C (Power Delivery) : La présence de deux résistances de 5,1 kΩ sur les broches CC1 et CC2 du connecteur USB-C est une sécurité matérielle importante. Elles forcent les chargeurs modernes à délivrer uniquement le profil 5V standard et empêchent ainsi l'envoi de tensions supérieures qui détruiraient l'électronique de la carte.

- Gestion thermique et électrique : Le régulateur AMS1117 dissipe la chaleur liée à la conversion de tension (5V vers 3,3V) grâce à sa large languette soudée directement sur le plan de masse, qui agit comme un radiateur passif. En parallèle, des condensateurs de découplage placés stratégiquement absorbent les pics de courant lors des frappes polyphoniques, protégeant le système des variations brutales de tension.

Conclusion et Perspectives

Ce projet d'électronique nous a permis de traverser l'intégralité du cycle de développement d'un produit électronique embarqué : de la rédaction du cahier des charges jusqu'à la démonstration du prototype assemblé, en passant par la modélisation mathématique du son, le développement logiciel en C, la conception du schéma sous KiCad, la fabrication industrielle du PCB et l'assemblage manuel.

Sur le plan technique, l'algorithme de Karplus-Strong s'est avéré un choix judicieux : il est simple à comprendre et à implémenter et il produit un rendu sonore naturel évoquant une corde frappée, tout en restant parfaitement calculable en temps réel sur un STM32 cadencé à 80 MHz. La gestion des interruptions via TIM4 garantit la stabilité de la fréquence d'échantillonnage à 44 100 Hz, condition indispensable à la justesse des notes.

Sur le plan de la gestion de projet, nous avons appris à travailler en équipe de six en maintenant une progression parallèle sur les blocs matériel et logiciel, à respecter des délais contraints, et à prendre des décisions collectives.

Chacun d'entre nous a développé de nouvelles compétences : maîtrise de KiCad et des outils de CAO électronique, programmation embarquée en C avec gestion du temps réel, traitement du signal audio, et coordination de projet en groupe pluridisciplinaire.

Perspectives d'amélioration

Extension du clavier : L'ajout d'un composant multiplexeur sur le circuit imprimé permettrait de contourner la limite actuelle des broches GPIO du STM32, afin de gérer un clavier complet de plusieurs octaves.

Connectivité MIDI (USB et Bluetooth) : L'implémentation du protocole MIDI transformerait le piano en un véritable contrôleur musical. Une idée initialement envisagée par l'équipe était d'ailleurs l'ajout d'un module Bluetooth Low Energy (BLE) pour transmettre ces données MIDI sans fil et sans latence (contrairement au Bluetooth audio) vers un smartphone ou une tablette.

Conseils pour les futurs projets

Conseils positifs

- Avancer en parallèle sur le matériel et le logiciel : Ne pas attendre d'avoir le circuit imprimé final pour commencer à coder.
- Imprimer le PCB sur papier à l'échelle 1:1 : Avant d'envoyer les fichiers à la Factory, imprimer le circuit sur une feuille et y poser physiquement les vrais composants. Cela permet de détecter les erreurs d'empreinte (trous trop petits, composants qui se chevauchent) impossibles à corriger plus tard.

- Valider les algorithmes sur PC en premier lieu : Tester une logique mathématique complexe (comme notre génération sonore) d'abord en Python ou en C permet de vérifier les calculs en quelques secondes (le débogage sur microcontrôleur est beaucoup plus lourd).

Points à éviter

- Commander les composants à la dernière minute : Les distributeurs (Mouser, Farnell) ont parfois des délais incompressibles ou des ruptures de stock.

- Négliger le plan de masse sur le routage : Un plan de masse mal relié sur KiCad peut entraîner des parasites (bruit de fond, instabilité des capteurs) impossibles à rattraper une fois la carte fabriquée.

- Faire confiance au code sans vérifier le matériel : Avant de passer beaucoup de temps à chercher une erreur dans son code C, il faut vérifier à l'oscilloscope que le matériel réagit bien comme prévu (fréquences des Timers, signaux GPIO).

Piano Numérique — Clavier une Octave

Mini-Projet d'Électronique 1A — ENSEA — 2025/2026

Conception et réalisation d'un piano numérique d'une octave (Do à Do, 13 touches) basé sur la carte Nucleo-L476RG (STM32L476). Génération sonore en temps réel par l'algorithme de Karplus-Strong implémenté en C. Sortie audio via amplificateur de Classe D PAM8403 (3 W), haut-parleur intégré et prise Jack 3,5 mm. Alimentation USB-C avec régulation AMS1117-3.3. PCB deux couches conçu sous KiCad, assemblé manuellement.

NORI Widad — CORBIÈRE Élie — FAUVEL Julien — ROUPENEL Léo — M'TIR Wissem —
MESTRE PINCHO Claire

Encadrant : M. Nicolas SIMOND | Fév. – Mai 2026 | 40 heures